

大模型微调、部署与前后端调用全流程解析

从模型选型到生产上线：一位AI开发者的实践指南

本次分享大纲：我们的探索路径

01. 启程：为何需要“定制”大模型？

技术背景与商业动机



02. 炼金：核心微调技术深度解析

SFT, RAG与LoRA



03. 备料：框架选型与环境搭建

工具与基础设施



04. 铸剑：端到端部署实战演练

从模型到API服务



05. 论道：全流程总结与展望

核心要点回顾



企业需要个性化大模型的四大驱动力



需求多样性

企业追求大模型定制化，以满足特定业务场景，如客服对话、产品推荐等。



性能与成本

平衡模型精度与计算资源，寻求高能效比，降低运营成本。



隐私与安全

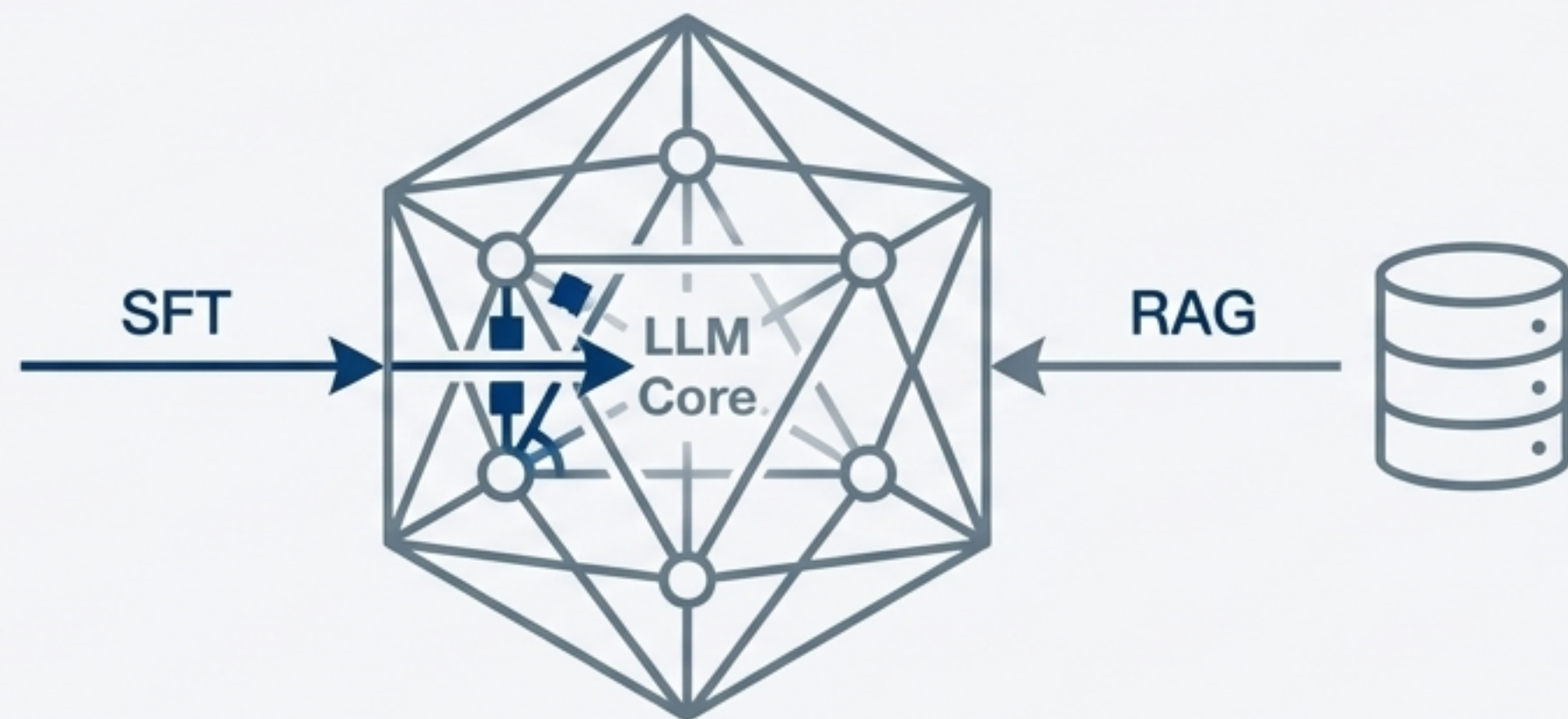
保护敏感信息，确保数据安全，遵守行业法规，构建信任。



可扩展性

设计灵活架构，支持未来技术升级，适应市场变化。

主流大模型个性化技术概览



SFT（有监督微调）

通过标注数据指导模型调整，提升特定任务表现，如分类或问答。



RLHF（强化学习）

利用人类反馈来奖励机制训练模型，优化生成文本的质量和相关性，适用于对话系统。



RAG（检索增强生成）

结合外部信息源，增强模型生成能力，提供更准确、详细的信息响应。

关键抉择：微调（Fine-tuning）还是RAG？

理解微调（Understanding Fine-tuning）



核心

教会模型新的“技能”或“风格”。它改变模型内部的权重以适应特定任务。

优势

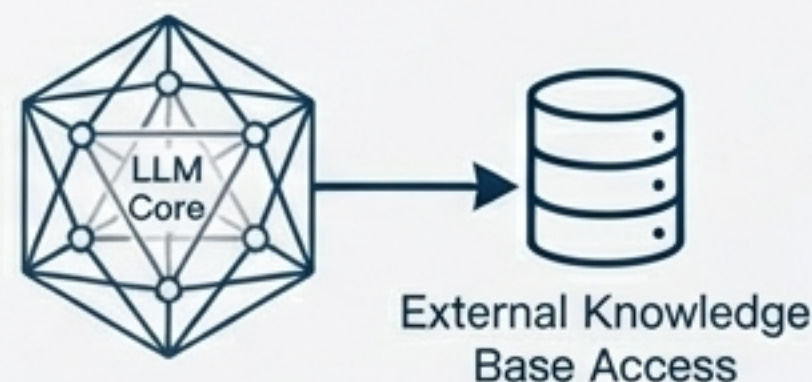
能够提升模型对特定场景的理解能力，深度嵌入领域知识。

适用场景

需要改变模型应答方式、语气或格式，且已有大量高质量标注数据时。



RAG优势解析（The RAG Advantage）



核心

给模型提供新的外部“知识库”。它不改变模型本身，而是提供实时信息。

优势

知识更新方便快捷，有效减少“幻觉”，特别适用于信息频繁更新的场景，如客服问答系统。

适用场景

需要模型回答基于动态、实时变化的数据时。

LoRA: 高效微调的“外挂”神器

核心思想

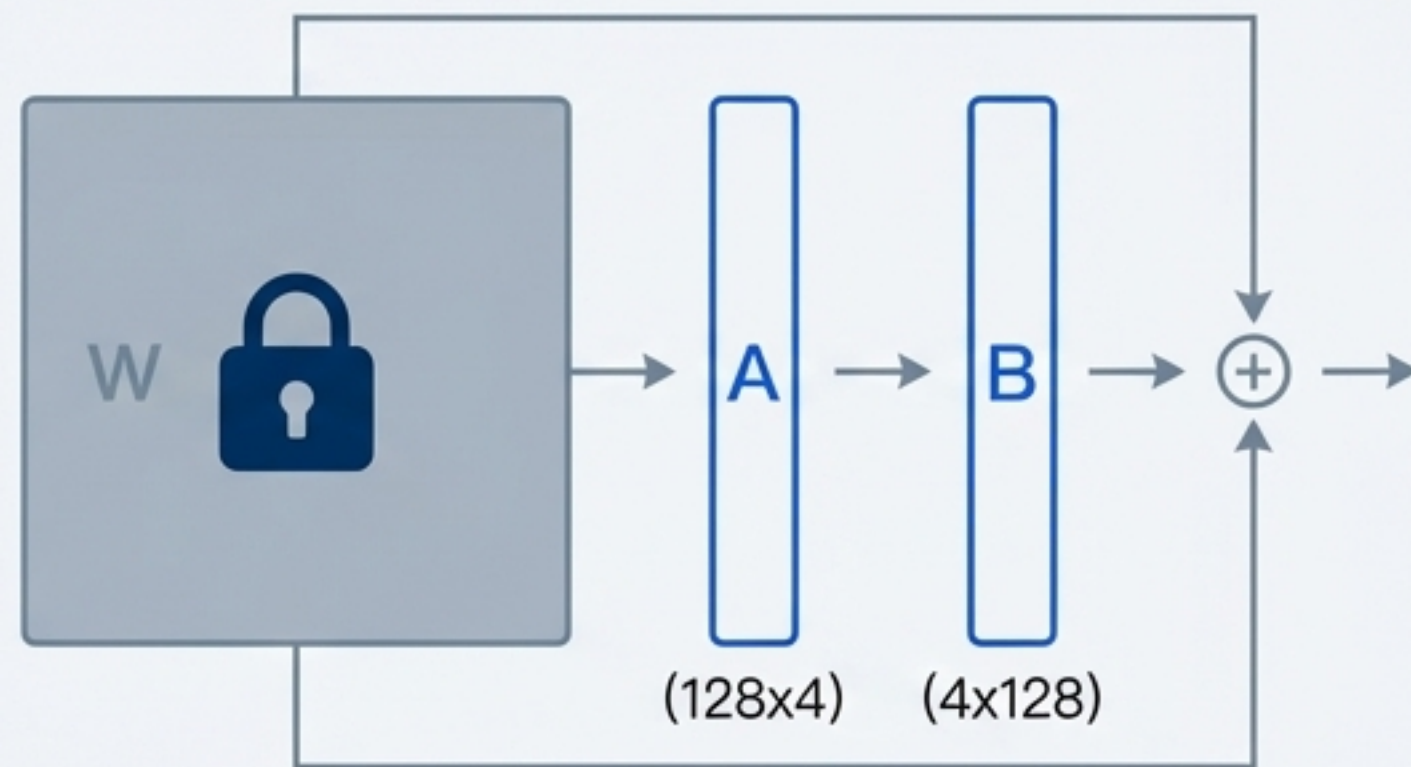
冻结预训练模型的绝大部分参数，仅引入并训练少量、低秩的“适配器”矩阵。通过这种方式，以极小的计算成本实现模型的高效个性化调整。

1. 轻量级微调

LoRA通过引入低秩矩阵来更新模型的局部权重，有效减少计算成本和存储需求。

4. 实际应用广泛

在特定任务场景下，如情感分析或命名实体识别，通过LoRA微调少量参数就能显著提升模型性能。



$$W' \approx W + B * A$$

2. 个性化调整

特别适用于大规模预训练模型的个性化调整，能够针对性地调整模型的关键参数。

3. 加速微调过程

加速微调过程并显著降低对额外资源的需求，是实现高效模型定制的理想选择。

主流微调实现框架对比



Llama-Factory

特性：专注于高效微调，支持LoRA等算法，易于集成，加速模型训练与部署。对用户友好。



transformers.Trainer

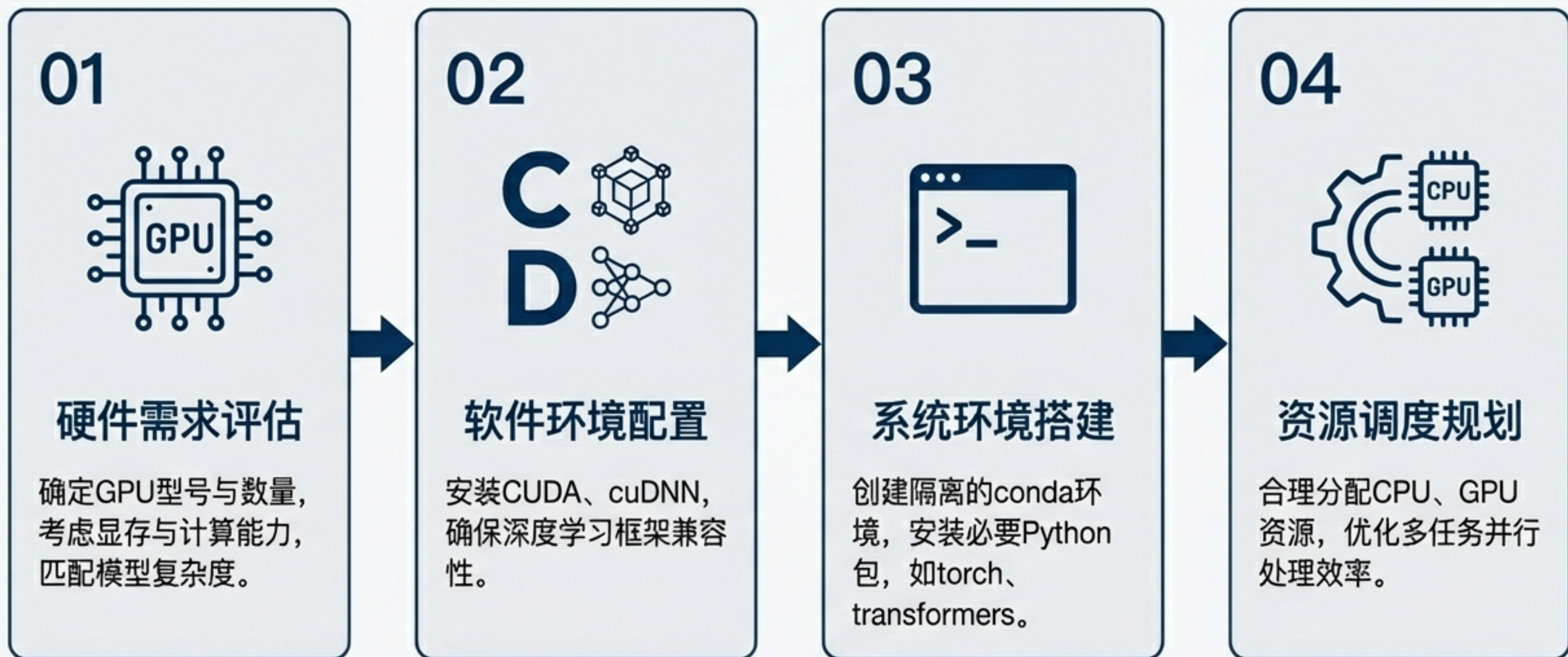
特性：来自Hugging Face，提供丰富的预训练模型，简化训练流程，适应多种任务。是行业标准。



DeepSpeed

特性：微软研发，专攻大规模模型训练，优化内存使用，显著提升训练速度与效率。适用于专家级大规模训练。

准备硬件资源及搭建开发环境



SFT实战流程：从数据到模型的五大步骤

Step 1: 收集领域文本

企业收集了大量特定领域的文本数据。



Step 2: 标注文本数据

对收集到的文本进行详细标注，提高数据质量。格式通常为`{"instruction": "...", "input": "...", "output": "..."}`。



Step 3: 微调预训练模型

使用标注后的数据对预训练模型进行微调。



Step 4: 提升模型表现

经过微调，模型在特定任务上的表现显著提升。



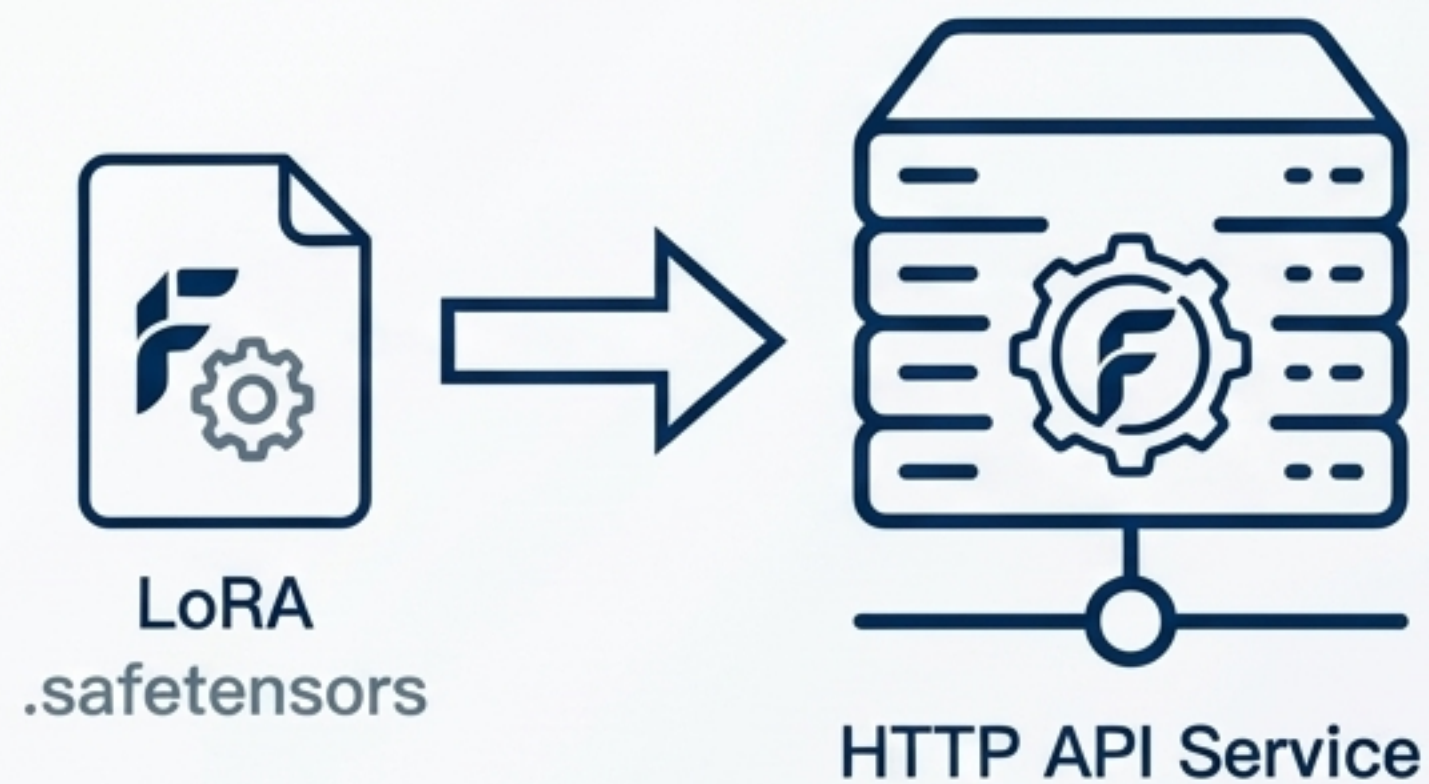
Step 5: 实现业务目标

模型性能的提升有助于企业更好地达成业务目标。

使用LLaMA-Factory进行模型微调的操作流程



模型部署：封装为可调用的HTTP服务



核心工具 (Key Tool)

FastAPI – 一个现代化、高性能的Python Web框架。

为何选择FastAPI? (Why FastAPI?)

- 异步支持：完美匹配I/O密集型的模型推理任务。
- 自动文档：自动生成交互式API文档 (Swagger UI)。
- 高性能：基于Starlette和Pydantic，性能卓越。

部署四步曲 (4-Step Deployment Process)

1. 环境准备：`pip install fastapi uvicorn`，创建隔离的conda虚拟环境。
2. 模型加载：在代码中导入模型，使用预训练权重初始化，准备模型部署。
3. 接口设计：定义HTTP POST接口，接收输入数据，调用模型预测，返回结果。
4. 启动服务：运行FastAPI应用，监听指定端口，确保外部可访问。

利用FastAPI开放HTTP接口：代码实例

```
# main.py
from fastapi import FastAPI
from pydantic import BaseModel
from transformers import AutoModelForCausalLM, AutoTokenizer

# 1. 加载微调后的模型和分词器
tokenizer = AutoTokenizer.from_pretrained("./output_model")
model = AutoModelForCausalLM.from_pretrained("./output_model").to("cuda")

app = FastAPI()

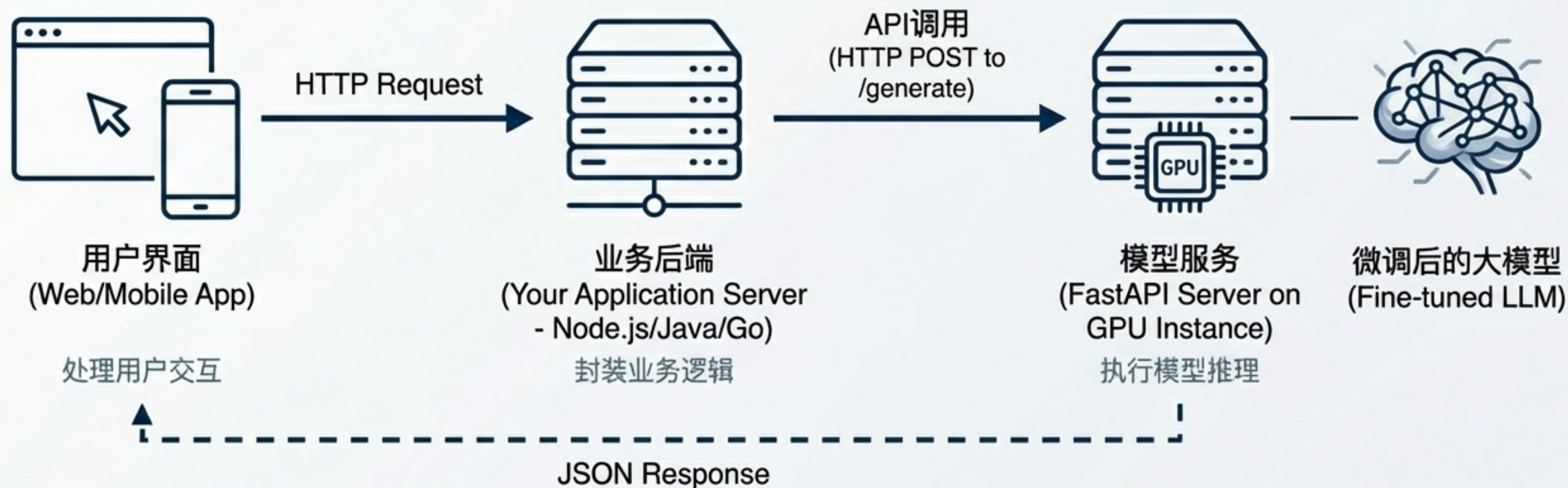
# 2. 定义请求体结构
class Item(BaseModel):
    prompt: str

# 3. 创建预测端点
@app.post("/generate/")
async def generate(item: Item):
    inputs = tokenizer(item.prompt, return_tensors="pt").to("cuda")
    outputs = model.generate(**inputs, max_new_tokens=128)
    response = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return {"response": response}
```

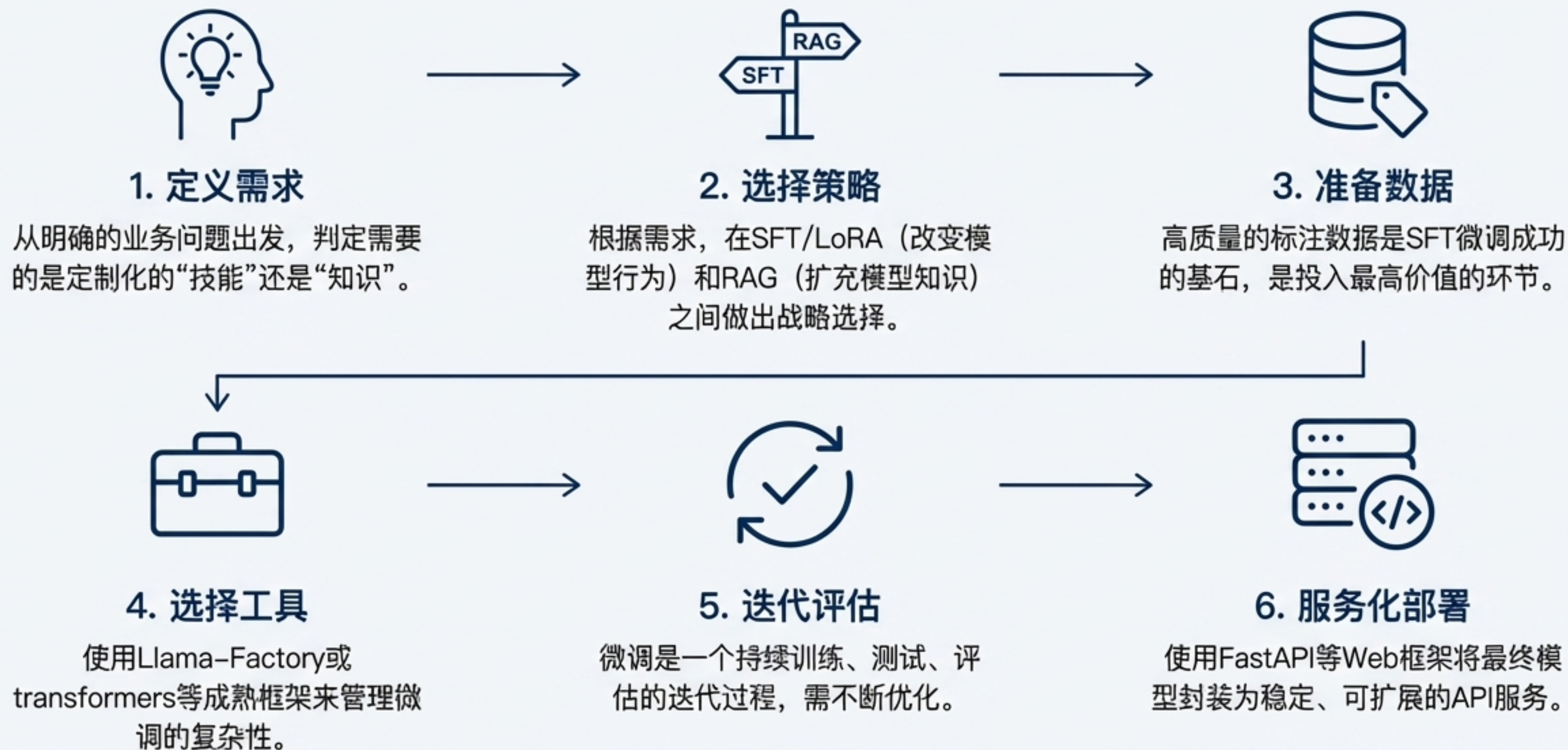
启动命令 (Execution Command) :

```
$ uvicorn main:app --host 0.0.0.0 --port 8000
```

端到端调用架构示意图



总结：端到端全流程核心要点回顾



Q&A

提问与交流



contact@example.com



github.com/username